

Performance Optimization Report

1. Identified Bottlenecks

During the initial analysis and code inspection, several bottlenecks were identified:

- **Inefficient Product Lookups:** The code relies on a custom binary tree for product lookups, but it performs a linear search in some cases (e.g., `productRepository.findById`). Additionally, no caching was implemented, causing repeated database lookups for frequently accessed data.
- **Concurrency Issues:** The `BinaryTree` used for product categorization could face potential race conditions when multiple threads concurrently access or update the tree (e.g., inserting or deleting products).
- **Unoptimized Tree Updates:** When updating products, the product was deleted from the tree and reinserted. This operation is less efficient compared to updating the existing node directly in the tree.
- **Lack of Caching:** There was no caching mechanism to optimize database access for frequently accessed queries such as fetching all products, paginated results, or searching by product attributes.
- **Redundant Binary Tree Initialization:** The binary tree was initialized every time the `ProductService` class was instantiated, potentially slowing down application startup, especially as the number of products grows.
- **Synchronized Blocks:** Tree updates and deletions were unsynchronized, creating potential issues when accessed by multiple threads simultaneously, especially under high loads.

2. Performance Improvements Brought to the Code

- **Caching:** Implemented caching for frequently accessed data using Spring's caching mechanism (`@Cacheable`, `@CacheEvict`). This significantly reduces the load on the database by storing results of queries in memory.
- **Tree Update Efficiency:** Replaced the redundant delete-and-reinsert approach for product updates in the binary tree with a direct `update`

method. This reduces the overhead associated with modifying the tree structure.

- **Synchronized Tree Updates:** Added `synchronized` blocks to ensure that tree updates and deletions occur in a thread-safe manner, preventing race conditions in concurrent environments.
- **Lazy Initialization of Tree:** Ensured that the binary tree is only initialized once when the `ProductService` starts up by using a flag (`isTreeInitialized`). This avoids unnecessary re-initialization and speeds up application startup.
- **Indexing Key Fields:** Database indexes were added to frequently queried fields to optimize for data retrieval, data look up without requiring full table scans

3. Before and After Optimization

Metric	Before Optimization	After Optimization
Database Queries	Multiple redundant queries on repeated method calls (e.g., <code>findAll</code> , <code>findById</code>)	Reduced due to caching (e.g., <code>@Cacheable</code>)
Binary Tree Updates	Delete and reinsert on product update	Direct tree node updates without deletion
Tree Initialization	Always reinitialized when <code>ProductService</code> instantiated	Initialized only once, reducing startup time
Concurrency Handling	No synchronization for tree modifications	Synchronized methods for safe concurrent access
Performance Under Load	Slower due to repeated DB queries and race conditions	Faster, due to caching and synchronized operations
Search Time for Existing Products	Slower linear search when querying DB multiple times	Faster, with binary tree lookup and cached results

4. Application's Adherence to 12-Factor Principles

I. Codebase:

The application adheres to a single codebase principle, with one code repository tracked via version control. The codebase is deployed across multiple environments but remains consistent.

II. Dependencies:

Dependency management is handled through Maven, ensuring all dependencies are explicitly declared in the `pom.xml`. This makes the build environment reproducible.

III. Config:

Configuration (e.g., database connection strings, cache settings) is stored in external configuration files (`application.properties`), keeping it separate from the codebase. This adheres to the 12-factor principle of strict separation between code and configuration.

IV. Backing Services:

The application treats the database and other external services (such as caching systems) as attached resources. Swapping one resource for another (e.g., a different database) can be done without modifying the codebase, adhering to this principle.

V. Build, Release, Run:

The application follows a clear distinction between the build, release, and run stages. After compiling and packaging the application (build), configurations are applied during deployment (release), and the application can be run without further changes.

VI. Processes:

The application is stateless. It doesn't rely on local state persistence and instead uses external systems (like the database and cache) for storage.

VII. Port Binding:

Spring Boot applications bind to ports to handle incoming requests, following the 12-factor guideline of exposing services via port binding.

VIII. Concurrency:

The application supports the ability to scale horizontally by handling multiple concurrent requests. However, concurrency issues were resolved by synchronizing tree modifications and implementing thread-safe mechanisms.

IX. Disposability:

The application can start and stop rapidly, particularly after optimizing the tree initialization process. With the optimizations, restarts are faster, and initialization of resources happens only once.

X. Dev/Prod Parity:

The development, staging, and production environments use the same stack and services, ensuring minimal divergence across environments. This helps maintain high parity between development and production environments.

XI. Logs:

Logs are managed externally by Spring Boot's logging mechanism, which writes to stdout or external log management tools, adhering to the 12-factor principle of treating logs as event streams.

XII. Admin Processes:

Management tasks (like database migrations, tree re-initialization, etc.) are scriptable and can be run as one-off admin processes, following the 12-factor guideline of treating admin processes as one-off tasks.